

MNIST DNN Implementation Project

2020311400 박세빈

PyTorch를 기반으로 한 DNN 구현, training과 dataset 불러오기를 위해 library들을 import

Libraries

```
#Import Libraries
import torch
from torch import nn, optim
import torch.nn.functional as F
import torchvision
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import numpy as np
```

Data Loading

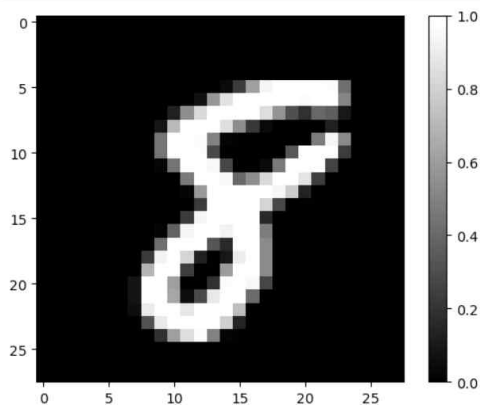
```
#Data Loading
batch_size = 12
train_data = datasets.MNIST('./datasets', train = True, download = True, transform = transforms.ToTensor())
test_data = datasets.MNIST('./datasets', train = False, download = True, transform = transforms.ToTensor())
train_loader = torch.utils.data.DataLoader(train_data, batch_size = batch_size, shuffle = True)
test_loader = torch.utils.data.DataLoader(test_data, batch_size = batch_size)
print(len(train_data))
print(len(test_data))

60000
10000
```

MNIST의 dataset을 불러오는 코드이다. batch size를 12로 설정해 training과 test 모두 12개씩 처리하도록 했다. training dataset은 60000개 test dataset은 10000개임을 알 수 있다.

Displaying MNIST Sample image

```
def imshow(img, colorbar=False):
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1,2,0)), cmap='gray')
    if colorbar:
        plt.colorbar()
    plt.show()
x, _train_data[777]
imshow(x, colorbar = True)
```



MNIST training dataset에서 777번째 이미지를 불러와 시각화하였다. 숫자 8 그림의 오른쪽을 보면 0~1까지 pixel intensity를 나타낸 bar도 출력했다. 0에 가까우면 픽셀이 검은색에 가깝고 1에 가까우면 픽셀은 하얀색에 가까운 것이다.

Model 1 - Large scale Model

Large-scale Model

```
18]: # Large scale model
class MLP_Large(nn.Module):
    def __init__(self):
        super().__init__()

        self.in_dim = 28*28
        self.out_dim = 10

        self.fc1 = nn.Linear(self.in_dim, 512)
        self.fc2 = nn.Linear(512,256)
        self.fc3 = nn.Linear(256,128)
        self.fc4 = nn.Linear(128,64)
        self.fc5 = nn.Linear(64, self.out_dim)

    def forward(self,x):
        x = F.relu(self.fc1(x.view(-1, self.in_dim)))
        x = F.relu(self.fc2(x))
        x = F.relu(self.fc3(x))
        x = F.relu(self.fc4(x))
        x = self.fc5(x)

        return x
```

첫 번째로 large-scale model을 구현했다. 5개의 Linear layer로 구성된 MLP 모델이다. 각 층은 fully connected layer로 구성되어 있고 각 hidden layer에는 ReLu activation function이 적용된다. 총 4개의 hidden layer와 1개의 output layer가 있는데 hidden layer의 neuron 수는 각각 512, 256, 128, 64개이고 마지막 output layer에서 10개의 neuron을 사용했다. 이 모델의 총 neuron은 $512+256+128+64+10 = 970$ 개가 사용되었다. 2의 제곱수를 neuron 수로 사용한 이유는 대부분의 하드웨어는 2의 배수 단위 데이터를 처리하는데 최적화되어 있고 neuron 수를 자연스러운 피라미드 구조로 설계 가능해 점진적으로 늘리거나 줄일 수 있다.

Training dataset using Large-scale Model

```
#Training dataset using Large scale model
model1 = MLP_Large()
criterion = nn.CrossEntropyLoss() # Loss function으로 CrossEntropyLoss 설정
optimizer = optim.SGD(model1.parameters(), lr = 0.01) #모델의 학습을 위해 SGD optimizer 설정 학습률:0.01
train_losses = [] # training Loss 저장 리스트
for epoch in range(5):
    running_loss = 0.0
    cost = 0.0
    for i,data in enumerate(train_loader, 0): #학습 데이터를 배치 단위로 가져옴
        inputs, labels = data
        model1.zero_grad() #모델의 기울기 초기화

        outputs = model1(inputs) #모델에 inputs를 넣어 예측 결과 생성
        loss = criterion(outputs, labels) #모델의 예측 결과 outputs와 labels를 비교해 Loss 계산
        loss.backward() # Loss를 기준으로 gradient 계산
        optimizer.step() # 계산된 gradient 사용해 Loss function 업데이트
        cost += loss.item()
        running_loss += loss.item()
        if (i+1) % 2000 == 0: # 2000번째 배치마다 손실을 출력
            print('[%d, %5d] loss: %.3f' %
                  (epoch + 1, i + 1, running_loss / 2000))
            running_loss = 0.0
    epoch_loss = cost / len(train_loader) #에포크 끝날때마다 평균 손실 저장
    train_losses.append(epoch_loss)

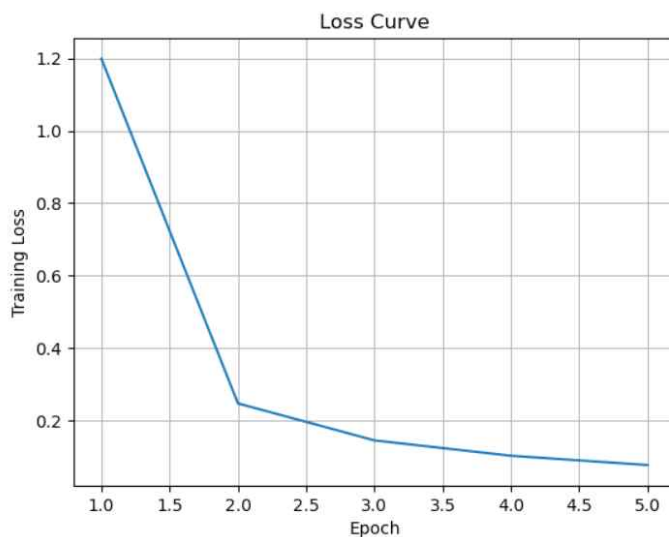
print('Finished Training')

[1, 2000] loss: 2.145
[1, 4000] loss: 0.651
[2, 2000] loss: 0.290
[2, 4000] loss: 0.228
[3, 2000] loss: 0.152
[3, 4000] loss: 0.141
[4, 2000] loss: 0.110
[4, 4000] loss: 0.099
[5, 2000] loss: 0.077
[5, 4000] loss: 0.078
Finished Training
```

이 코드는 앞서 만든 MLP_Large 모델을 이용해 MNIST 학습 데이터를 5 epoch 동안 학습시키는 코드다. CrossEntropyLoss를 손실 함수로, SGD optimizer를 사용했고 learning rate는 0.01로 설정했다. batch size가 12이기 때문에 5000개의 minibatch가 1 epoch에 사용된다. 위의 코드에서는 2000번째, 4000번째 배치에서의 loss를 출력하고 epoch마다 평균 손실을 epoch_loss에 저장하고 train_losses 리스트에 누적 했다.

Training loss graph(Large-scale model)

```
#PLOT
plt.plot(range(1, len(train_losses)+1), train_losses)
plt.xlabel('Epoch')
plt.ylabel('Training Loss')
plt.title('Loss Curve')
plt.grid()
plt.show()
```



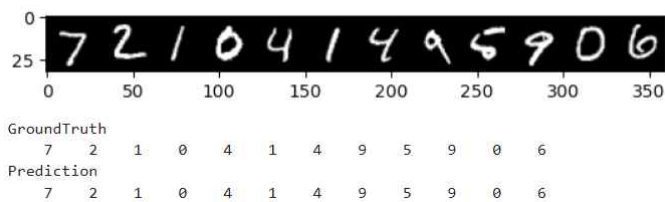
위의 large-scale model을 이용해 training했을 때의 loss 그래프를 plot했다. 위의 코드에서 출력된 값과 그래프의 y값인 training loss가 달라보이는 이유는 출력 로그에서 나타난 loss값은 일부 배치 기준의 loss지만 loss graph는 각 epoch 전체에 대한 평균 loss이기 때문에 값의 차이가 존재한다.

Test 12 data and show results

```
images, labels = next(iter(test_loader))

imshow(torchvision.utils.make_grid(images, nrow = batch_size))
print('GroundTruth')
print(" "+" ".join('%3s' %label.item() for label in labels))

outputs = model(images)
_, predicted = torch.max(outputs,1)
print("Prediction")
print(" "+" ".join('%3s' %label.item() for label in predicted))
```



이 코드는 test dataset에서 12개의 이미지를 시각화하고 모델이 예측한 결과와 실제 정답을 비교해서 출력하는 코드이다. imshow function이 이미지를 보여주고 GrounTruth가 test set의 label을 보여주고 Prediction이 train 결과를 보여준다. Large scale model을 사용해 test dataset의 12개의 이미지를 분류한 결과 모두 정확히 예측했음을 확인했다.

Test Accuracy

```
n_predict = 0
n_correct = 0

for data in test_loader:
    inputs, labels = data
    outputs = model1(inputs)
    _, predicted = torch.max(outputs, 1)

    n_predict += len(predicted)
    n_correct += (labels == predicted).sum()

print(f"{n_correct}/{n_predict}")
print(f"Accuracy: {n_correct/n_predict:.3f}")

9725/10000
Accuracy: 0.973
```

이 코드는 학습이 끝난 model의 Accuracy를 test dataset 기준으로 평가한다.

10000개의 test dataset을 사용하였다.

Test dataset 10000를 기반으로 평가한 결과 Large-scale model의 Accuracy는 0.973 즉 10000개중 9725개를 정확히 분류해 이에 따른 정확도는 97.3%를 기록했다.

Model 2-Medium Scale Model

Medium-scale Model

```
# Medium scale model
class MLP_Medium(nn.Module):
    def __init__(self):
        super().__init__()

        self.in_dim = 28*28
        self.out_dim = 10

        self.fc1 = nn.Linear(self.in_dim, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, self.out_dim)

    def forward(self, x):
        x = F.relu(self.fc1(x.view(-1, self.in_dim)))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)

        return x
```

Model 2는 Medium-scale Model이다. Large-scale Model과 차별을 두기 위해 layer를 2개 줄였다. 그래서 총 hidden layer 2개와 output layer 1개로 이루어진 MLP이다. Large-scale Model과 마찬가지로 ReLu를 각 hidden layer에서 activation function으로 사용했다. Hidden layer의 neuron수는 128, 64개이고 output layer의 neuron 수는 10개이다. 총 neuron 수는 $128 + 64 + 10 = 202$ 개이다.

Training dataset using Medium-scale Model

```
#Training dataset using Large scale model
model2 = MLP_Medium()
criterion = nn.CrossEntropyLoss() # loss function으로 CrossEntropyLoss 설정
optimizer = optim.SGD(model2.parameters(), lr = 0.01) # 모델의 학습을 위해 SGD optimizer 설정 학습률:0.01
train_losses = [] # training loss 저장 리스트
for epoch in range(5):
    running_loss = 0.0
    cost = 0.0
    for i, data in enumerate(train_loader, 0): # 학습 데이터를 배치 단위로 가져옴
        inputs, labels = data
        model2.zero_grad() # 모델의 기울기 초기화

        outputs = model2(inputs) # 모델에 inputs를 넣어 예측 결과 생성
        loss = criterion(outputs, labels) # 모델의 예측 결과 outputs와 labels를 비교해 loss 계산
        loss.backward() # loss를 기준으로 gradient 계산
        optimizer.step() # 계산된 gradient 사용해 loss function 업데이트
        cost += loss.item()
        running_loss += loss.item()
        if (i+1) % 2000 == 0: # 2000번째 배치마다 손실을 출력
            print('[%d, %d] loss: %.3f' %
                  (epoch + 1, i + 1, running_loss / 2000))
            running_loss = 0.0
    epoch_loss = cost / len(train_loader) # 에포크 끝날때마다 평균 손실 저장
    train_losses.append(epoch_loss)

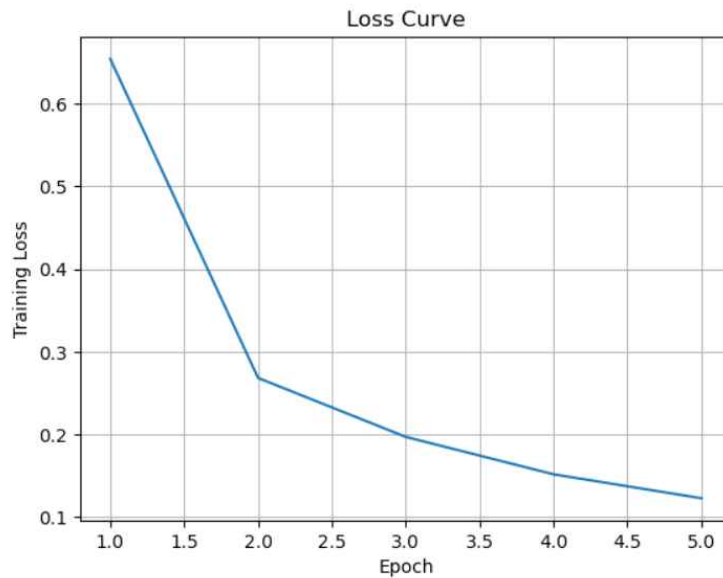
print('Finished Training')

[1, 2000] loss: 1.081
[1, 4000] loss: 0.388
[2, 2000] loss: 0.287
[2, 4000] loss: 0.259
[3, 2000] loss: 0.214
[3, 4000] loss: 0.191
[4, 2000] loss: 0.157
[4, 4000] loss: 0.150
[5, 2000] loss: 0.126
[5, 4000] loss: 0.121
Finished Training
```

MLP_Medium 모델을 이용해 MNIST 데이터를 동일하게 5 epoch 동안 training 했다. CrossEntropyLoss와 SGD optimizer도 사용했다. 2000번째, 4000번째 배치에서의 loss를 출력하고 epoch마다 평균 손실을 epoch_loss에 저장하고 train_losses 리스트에 누적 했다. 마지막 5 epoch째에서 4000번째 batch의 loss를 보면 0.121로 Large scale model 보다 loss 값이 큼을 알 수 있다.

Training loss graph(Middle-scale Model)

```
#PLOT
plt.plot(range(1, len(train_losses)+1), train_losses)
plt.xlabel('Epoch')
plt.ylabel('Training Loss')
plt.title('Loss Curve')
plt.grid()
plt.show()
```



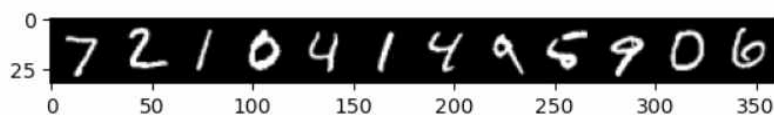
Medium-scale Model로 training 했을 때 측정된 loss의 값을 plot했다. 마찬가지로 epoch의 평균 loss를 계산한 것이기 때문에 위의 출력값과 다르게 나왔음을 확인할 수 있다. Medium-scale model은 초기 loss값은 Large-scale model 보다 낮지만 loss 감소 폭이 Large scale model에 비해 완만하게 수렴해 최종 loss값은 Large scale model보다 더 크다는 것을 확인했다.

Test 12 data and show results

```
images, labels = next(iter(test_loader))

imshow(torchvision.utils.make_grid(images, nrow = batch_size))
print('GroundTruth')
print(" '+' '.join('%3s' %label.item() for label in labels))

outputs = model(images)
_, predicted = torch.max(outputs,1)
print("Prediction")
print(" '+' '.join('%3s' %label.item() for label in predicted))
```



GroundTruth	7	2	1	0	4	1	4	9	5	9	0	6
Prediction	7	2	1	0	4	1	4	9	6	9	0	6

Medium-scale model은 1개의 mismatch가 발생하였다. 9번째 숫자의 정답인 GroundTruth가 5지만 trained model은 6으로 predict했음을 확인했다. 해당 MNIST 이미지를 확인해보면 숫자가 5지만 그림이 6과 시각적으로 유사하게 생겨 제대로 분류를 하지 못했다. epoch수를 늘려 학습을 늘리거나 data를 augmentation 해주거나 모델 구조를 개선하면 숫자 5를 제대로 예측할 수 있을 것이다.

Test Accuracy ¶

```
n_predict = 0
n_correct = 0

for data in test_loader:
    inputs, labels = data
    outputs = model2(inputs)
    _, predicted = torch.max(outputs, 1)

    n_predict += len(predicted)
    n_correct += (labels == predicted).sum()

print(f"{n_correct}/{n_predict}")
print(f"Accuracy: {n_correct/n_predict:.3f}")

9641/10000
Accuracy: 0.964
```

Test dataset 10000를 기반으로 평가한 결과 Medium-scale model의 Accuracy는 0.964 즉 10000개중 9641개를 정확히 분류해 이에 따른 정확도는 96.4%를 기록했다. Accuracy가 Model 1인 Large scale model보다는 소폭 감소했음을 확인할 수 있었다.

Model 3-Small scale model

Small-scale Model ¶

```
# Small scale model
class MLP_Small(nn.Module):
    def __init__(self):
        super().__init__()

        self.in_dim = 28*28
        self.out_dim = 10

        self.fc1 = nn.Linear(self.in_dim, 16)
        self.fc2 = nn.Linear(16, 8)
        self.fc3 = nn.Linear(8, self.out_dim)

    def forward(self, x):
        x = F.relu(self.fc1(x.view(-1, self.in_dim)))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)

        return x
```

Model 3는 Small-scale Model이다. Large scale, Medium scale Model보다도 경량화된 구조가 요구된다. layer 수를 조정하거나 neuron 수를 조정하거나 data type 변경을 통해 해결할 수 있다. 그러나 필요로 하는 model은 DNN model이기 때문에 더 이상의 layer를 줄이는 것은 불가하다. Hidden layer가 1개로 줄어들게 되면 Shallow Neural Network가 된다. 그렇기 때문에 hidden layer 2개와 output layer 1개로 이루어진 layer의 총개수는 같지만 neuron 수를 크게 감소시켰다. 기존 Model과 마찬가지로 ReLu를 각 hidden

layer에서 activation function으로 사용했다. Hidden layer의 neuron 수는 16, 8개이고 output layer의 neuron 수는 10개이다. 총 neuron 수는 $16 + 8 + 10 = 34$ 개이다. 여기서 hidden layer의 neuron 개수가 8개로 output layer의 neuron 수인 10개보다 적어서 의문을 가질 수 있다. Hidden layer의 neuron 개수가 output layer보다 적어도 문제 없다. 왜냐하면 hidden layer는 입력 데이터의 특징을 추상화하는 역할을 한다. 모델의 표현력을 조정하는 것과 관련이 있기 때문에 분류 클래스 수인 output layer의 neuron 수와 독립적으로 설계가 가능하다.

Training dataset using Small-scale Model

```
#Training dataset using Large scale model
model3 = MLP_Small()
criterion = nn.CrossEntropyLoss() # loss function으로 CrossEntropyLoss 설정
optimizer = optim.SGD(model3.parameters(), lr = 0.01) #모델의 학습을 위해 SGD optimizer 설정 학습률:0.01
train_losses = [] # training loss 저장 리스트
for epoch in range(5):
    running_loss = 0.0
    cost = 0.0
    for i,data in enumerate(train_loader, 0): #학습 데이터를 배치 단위로 가져옴
        inputs, labels = data
        model3.zero_grad() #모델의 기울기 초기화

        outputs = model3(inputs) #모델에 inputs를 넣어 예측 결과 생성
        loss = criterion(outputs, labels) #모델의 예측 결과 outputs와 labels를 비교해 loss 계산
        loss.backward() # loss를 기준으로 gradient 계산
        optimizer.step() # 계산된 gradient 사용해 loss function 업데이트
        cost += loss.item()
        running_loss += loss.item()
        if (i+1) % 2000 == 0: # 2000번째 배치마다 손실을 출력
            print('[%d, %5d] loss: %.3f' %
                  (epoch + 1, i + 1, running_loss / 2000))
            running_loss = 0.0
    epoch_loss = cost / len(train_loader) #에포크 끝날때마다 평균 손실 저장
    train_losses.append(epoch_loss)

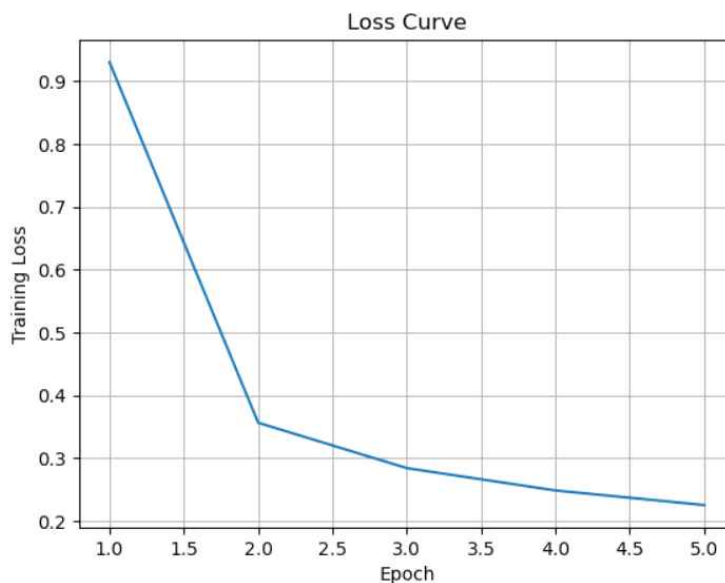
print('Finished Training')

[1, 2000] loss: 1.547
[1, 4000] loss: 0.555
[2, 2000] loss: 0.390
[2, 4000] loss: 0.341
[3, 2000] loss: 0.295
[3, 4000] loss: 0.276
[4, 2000] loss: 0.249
[4, 4000] loss: 0.249
[5, 2000] loss: 0.240
[5, 4000] loss: 0.211
Finished Training
```

MLP_Small model을 이용해 training 했을 때 loss 값을 출력해냈다. 마찬가지로 2000, 4000번째마다 출력을 했고 loss 값을 console에 보이게 만들었다. 5 epoch가 지나서 4000번째 배치의 loss가 0.211로 large scale, medium scale model보다 더 크다는 것을 확인했다.

Training loss graph(Small-scale Model) ¶

```
#PLOT
plt.plot(range(1, len(train_losses)+1), train_losses)
plt.xlabel('Epoch')
plt.ylabel('Training Loss')
plt.title('Loss Curve')
plt.grid()
plt.show()
```



Small-scale Model로 training 했을 때 측정된 loss의 값을 plot했다. Small-scale의 첫 epoch loss는 약 0.95정도로 Medium보다는 크고 Large보다는 작게 나왔다. 일정한 감소를 보이며 점진적으로 수렴했는데 최종 loss 수렴값이 Medium, Large scale model보다 큰 값이 나와 성능이 다른 모델들에 비해 낮다는 것을 확인할 수 있다.

Test 12 data and show results

```
images, labels = next(iter(test_loader))

imshow(torchvision.utils.make_grid(images, nrow = batch_size))
print('GroundTruth')
print(" "+" ".join('%3s' %label.item() for label in labels))

outputs = model(images)
_, predicted = torch.max(outputs,1)
print("Prediction")
print(" "+" ".join('%3s' %label.item() for label in predicted))
```



GroundTruth	7	2	1	0	4	1	4	9	5	9	0	6
Prediction	7	2	1	0	4	1	4	9	6	9	0	6

Small scale model도 Medium scale model과 동일하게 9번째 숫자인 5에서 mismatch가 발생하였다. 더 복잡한 model도 오류가 발생하였는데 경량화된 model이 해당 숫자를 정확하게 인식한다는 것은 더 어려울 것으로 판단된다.

▼ Test Accuracy

```
9]: n_predict = 0
    n_correct = 0

    for data in test_loader:
        inputs, labels = data
        outputs = model3(inputs)
        _, predicted = torch.max(outputs, 1)

        n_predict += len(predicted)
        n_correct += (labels == predicted).sum()

    print(f"{n_correct}/{n_predict}")
    print(f"Accuracy: {n_correct/n_predict:.3f}")

9352/10000
Accuracy: 0.935
```

Test dataset 10000를 기반으로 평가한 결과 Small-scale model의 Accuracy는 0.935 즉 10000개중 9352개를 정확히 분류해 이에 따른 정확도는 93.5%를 기록했다. Accuracy가 Model 1과 Model 2에비해 큰 폭으로 감소했음을 확인할 수 있었다.

Model1_Quant8 - Large-scale model with Quantization

Large-scale model with Quantization

```
#Pytorch 공식 이용해서 Dynamic Quantization 적용
model1_quant8 = torch.quantization.quantize_dynamic(
    model1, #학습 완료된 모델
    {torch.nn.Linear}, #Linear 계층만 양자화
    dtype=torch.qint8 #8비트 정수 양자화
)
```

3개중 1개의 model을 골라 Quantization하기로 했는데 Large-scale model을 이용하기로 했다. 처음에는 Small-scale model에 적용하려고 했지만 Accuracy 측정을 해보니 Quantization된 model이 오히려 기존 model보다 accuracy가 올라가는 경우가 생겨서 Large-scale model을 채택하였다. Quantization이 적용된 Small scale model의 accuracy가 기존 model보다 높았던 이유를 생각해보면 Quantization은 parameter를 정수로 근사하면서 model의 표현력을 줄이기 때문에 Regularization 효과를 가진다. 특히 Small scale model은 파라미터 수 자체가 적기 때문에 weight값의 분포가 단순하고 scale도 작다. 그래서 float -> int로 변환 시 precision 손실이 작아 Quantization 오차가 상대적으로 덜 치명적이다. 반대로 Large scale model은 구조가 복잡하기 때문에 Quantization 오차가 누적되면 성능에 더 큰 영향을 미칠 수 있다. Quantization은 여러 방법이 존재한다. 직접 Quantization해서 학습 끝난 model의 parameter를 8bit범위로 round해서 다시 저장하는 방법, Quantization Aware Training(QAT) 그리고 Dynamic Quantization 방법 등이 존재한다. 직접 Quantization하는 방법은 PyTorch에서 지원하는 방식이 아니고 QAT는 좋은 기술이지만 지금 프로젝트에 적용하기에는 과해서 Dynamic Quantization 방법을 채택했다. Dynamic Quantization 방식은 Training이 완료된 후 적용하는 Quantization 방법이다. 학습 중이 아닌 forward 할 때 weight와 연산의 정수화를 진행한다. 특히 Linear 계층이 많은 모델에 적합한 방식이다. PyTorch의

quantize_dynamic() function을 이용해 Quantization을 적용했다. Quantization 대상은 nn.Linear 계층이고 parameter는 32bit float에서 torch.qint8(8비트 정수)로 변환되었다.

Test 12 data and show results

```
images, labels = next(iter(test_loader))

imshow(torchvision.utils.make_grid(images, nrow = batch_size))
print('GroundTruth')
print(" '+' '.join('%3s' %label.item() for label in labels))

outputs = model1_quant8(images)
_, predicted = torch.max(outputs,1)
print("Prediction")
print(" '+' '.join('%3s' %label.item() for label in predicted))
```



GroundTruth										
7	2	1	0	4	1	4	9	5	9	0
6										

Prediction										
7	2	1	0	4	1	4	9	5	9	0
6										

Dynamic Quantization이 적용된 Large-scale model을 사용해 12개의 test data 예측 결과를 확인했다. Quantization이 적용되지 않은 기존 model과 동일하게 모든 숫자를 정확히 예측했다.

Test Accuracy

```
: n_predict = 0
  n_correct = 0

  for data in test_loader:
      inputs, labels = data
      outputs = model1_quant8(inputs)
      _, predicted = torch.max(outputs, 1)

      n_predict += len(predicted)
      n_correct += (labels == predicted).sum()

  print(f"{n_correct}/{n_predict}")
  print(f"Accuracy: {n_correct/n_predict:.3f}")

9723/10000
Accuracy: 0.972
```

Test dataset 10000를 기반으로 평가한 결과 Dynamic Quantization이 적용된 Large-scale model의 Accuracy는 0.972 즉 10000개중 9723개를 정확히 분류해 이에 따른 정확도는 97.2%를 기록했다. Quantization을 적용하지 않은 model과 비교했을 때 10000 개중 2개 정도만 추가로 mispredict 했다는 것을 확인했고 정확도가 적은 폭으로 감소했음을 확인했다.

Result

	Model 1	Model1_Quant8	Model 2	Model 3
#of neuron	970	970	202	34
#of layer	5	5	3	3
Accuracy	97.3%	97.2%	96.4%	93.5%

결과를 분석해 보면 일단 Accuracy 비교부터 해보겠다. 우선 model의 size에 따라 Accuracy가 차이났음을 확인할 수 있다. Large scale model인 Model 1이 neruon의 개수가 970개 가장 많고 layer의 수가 5개로 가장 깊어 보다 복잡한 것까지 정밀하게 학습이 가능해 Accuracy가 97.3%로 가장 높았다. 반면 Small scale model인 Model 3은 neuron 수도 가장 적고, 깊이도 얇아 Accuracy가 93.5%로 가장 낮았다. Model 1과 Model 2를 비교해보면 neuron도 768개 줄고 layer도 5에서 3으로 줄었는데 Accuracy가 96.4%로 0.8% 소폭 감소하였다. Model 2와 Model 3를 비교해보면 layer는 같지만 neuron 수가 168개 줄어들었다. neuron 수의 변화로 Accuracy가 93.5%로 2.9%나 크게 감소하였다. 이를 통해 일정 개수 이상의 neuron이 존재하게 되면 Accuracy가 크게 줄어들지 않는다는 점을 추론해 볼 수 있다. 다음으로 Quantization에 관한 점을 살펴보겠다. Model1_Qunat8은 Model1을 기반으로 weight와 bias를 8비트 정수로 양자화한 model이다. 양자화로 인해 미세하게 weight 조정이 제한되어 약간의 Accuracy(0.1%)의 감소가 일어났다. 그러나 구조 자체는 Model1과 같기 때문에 여전히 높은 Accuracy를 보임을 알 수 있다.

각 모델의 장단점을 살펴보겠다.

Large scale model인 Model 1은 장점으로는 가장 높은 정확도를 가지고 있고 복잡한 것도 정밀하게 분류가 가능하다. 단점으로는 학습시간이 길고 메모리 사용량이 크다.

Model1_Quant8의 장점은 Model1과 비슷하게 높은 정확도를 유지하면서도 데이터 형식이 8비트 정수로 정의 되어 있기 때문에 필요한 메모리 공간이 적다. 그래서 경량화가 필요할 때 적합하다. 단점으로는 Accuracy가 소폭 감소할 수 있고 정밀도가 줄어들 수 있다.

Model 2의 장점으로는 학습 속도가 Model 1에 비해 빠르고 연산량과 정확도의 균형이 좋다. 단점으로는 복잡한 데이터에서는 정확도가 떨어질 수 있다.

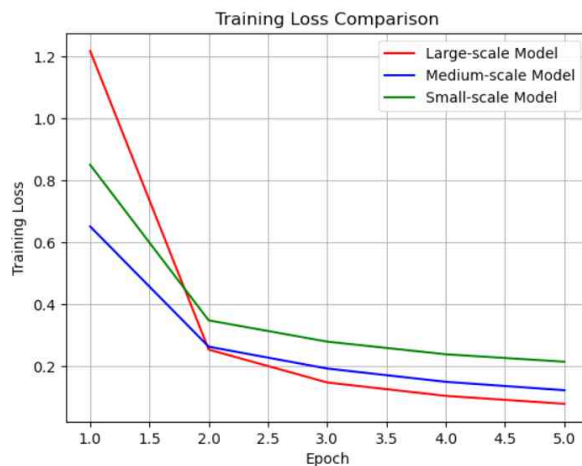
Model 3의 장점으로는 가장 빠른 학습 속도를 보여주고 메모리 사용량을 최소화 했다. 이런 장점으로 실시간 시스템 같은 곳에 활용할 수 있다. 단점으로는 가장 낮은 정확도를 보인다. 그래서 결론적으로 각각의 장단점이 존재하고 필요한 상황에 따라 적절한 model을 사용하면 된다. 예를 들어 데이터가 복잡하고 정확도가 중요한 상황에는 Model 1을 경량화가 필요한 상황에서는 Model3나 Model1_Quant8을 사용하면 된다. 일반적으로 Model이 크면 overfitting 위험이 크다. 하지만 이번 프로젝트에서는 학습 데이터 양이 충분하고 학습 설정이 적절했기 때문에 overfitting의 상황이 발생하기보다 오히려 더 안정적이고 정확도 높은 결과를 보여주었다. 이를 통해 overfitting을 억제하는 요인중에 학습 환경이 중요하다는

점도 추론할 수 있었다.

Three Loss graph ¶

```
plt.plot(range(1, len(train_losses_1)+1), train_losses_1, label='Large-scale Model', color='red')
plt.plot(range(1, len(train_losses_2)+1), train_losses_2, label='Medium-scale Model', color='blue')
plt.plot(range(1, len(train_losses_3)+1), train_losses_3, label='Small-scale Model', color='green')

plt.xlabel('Epoch')
plt.ylabel('Training Loss')
plt.title('Training Loss Comparison')
plt.legend()
plt.grid()
plt.show()
```



각 model마다 train_losses라는 변수에 _1,_2,_3을 다시 붙이고 training을 다시하여 한번에 세 개의 그래프를 plot해 비교할 수 있었다. 일단 모든 model이 epoch가 증가할수록 training loss가 점차 감소하면서 정상적으로 학습이 되었음을 확인할 수 있다. 각 Model별로 차이를 보면 우선 빨간색 그래프인 Large-scale Model은 초기 loss값이 가장 크고 1~2epoch사이에 급격한 loss 감소를 보이고 있다. 학습 초기에는 불안정하지만 결국 최종 loss값이 가장 낮음을 확인할 수 있다. 파란색 그래프인 Medium-scale Model은 초기 loss값이 낮고 전체적으로 가장 완만하고 안정적이다. 최종 loss값은 Large scale Model보다는 높고 Small scale Model보다는 낮다. 이를 통해 세 model중 균형 잡힌 model이라는 것을 알 수 있다. 마지막으로 초록색 그래프인 Small-scale Model은 초기 loss값이 Large보다는 낮고 Medium 보다는 높게 나왔다. loss 감소 폭이 가장 완만하고 최종 loss 값이 가장 높다. 이를 통해 model이 너무 단순해 학습은 안정적이지만 정확도가 가장 낮을 것이라는 결론을 도출 할 수 있었다.

마지막으로 양자화에 관해서 얘기해 보겠다. 이번 프로젝트에서는 32bit float를 8bit 정수로 줄였다. 양자화의 궁극적인 목표는 model의 크기와 연산량을 줄이면서도 최대한 정확도 손실을 최소화 시키는 것이다. 양자화 모델의 장점으로는 데이터 타입을 8bit 정수로 바꿨기 때문에 메모리를 절약할 수 있고 정수 연산이기 때문에 float 연산보다 추론속도가 더 빠르다. 그리고 전력 소모를 감소 시킬 수 있다. ARM, NVIDIA, Google Edge TPU와 같은 하드웨어들이 int8 최적화를 지원해 하드웨어 친화적이다. 이런 내용을 바탕으로 양자화된 모델은 낮은 전력 소모로 edge device에 적합한 모델임을 알 수 있다.

코드 실행 방식

주피터에서 Run all cell하면 잘 돌아갑니다.